

Estacion I2C Raspberry

Generated by Doxygen 1.17.0

1 Directory Hierarchy	1
1.1 Directories	1
2 Namespace Index	3
2.1 Package List	3
3 File Index	5
3.1 File List	5
4 Directory Documentation	7
4.1 webP/static Directory Reference	7
4.2 webP/templates Directory Reference	7
4.3 webP Directory Reference	7
5 Namespace Documentation	9
5.1 app Namespace Reference	9
5.1.1 Detailed Description	9
5.1.2 Function Documentation	10
5.1.2.1 index()	10
5.1.3 Variable Documentation	10
5.1.3.1 app	10
5.1.3.2 DB_NAME	10
5.1.3.3 debug	10
5.1.3.4 host	10
5.1.3.5 port	10
5.2 mqtt_sqlite Namespace Reference	11
5.2.1 Detailed Description	11
5.2.2 Function Documentation	12
5.2.2.1 guardar_exterior()	12
5.2.2.2 guardar_interior()	12
5.2.2.3 on_connect()	12
5.2.2.4 on_message()	13
5.2.3 Variable Documentation	13
5.2.3.1 BROKER	13
5.2.3.2 cliente	13
5.2.3.3 DB_NAME	13
5.2.3.4 on_connect	14
5.2.3.5 on_message	14
5.2.3.6 PORT	14
5.2.3.7 TOPIC_EXTERIOR	14
5.2.3.8 TOPIC_INTERIOR	14
5.3 thingspeak_uploader Namespace Reference	14
5.3.1 Detailed Description	15
5.3.2 Function Documentation	15

5.3.2.1 enviar_thingspeak()	15
5.3.2.2 obtener_ultimos_datos()	16
5.3.3 Variable Documentation	16
5.3.3.1 codigo	16
5.3.3.2 DB_NAME	16
5.3.3.3 exterior	16
5.3.3.4 exterior_nuevo	16
5.3.3.5 id_exterior	16
5.3.3.6 id_interior	16
5.3.3.7 interior	17
5.3.3.8 interior_nuevo	17
5.3.3.9 THINGSPEAK_API_KEY	17
5.3.3.10 THINGSPEAK_URL	17
5.3.3.11 ultimo_id_exterior	17
5.3.3.12 ultimo_id_interior	17
6 File Documentation	19
6.1 mqtt_sqlite.py File Reference	19
6.2 thingspeak_uploader.py File Reference	20
6.3 webP/app.py File Reference	20
6.4 webP/static/style.css File Reference	20
6.5 webP/templates/index.html File Reference	20
Index	21

Chapter 1

Directory Hierarchy

1.1 Directories

static	7
style.css	20
templates	7
index.html	20
webP	7
static	7
style.css	20
templates	7
index.html	20
app.py	20

Chapter 2

Namespace Index

2.1 Package List

Here are the packages with brief descriptions (if available):

app	9
mqtt_sqlite	11
thingspeak_uploader	14

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

mqtt_sqlite.py	19
thingspeak_uploader.py	20
webP/app.py	20
webP/static/style.css	20
webP/templates/index.html	20

Chapter 4

Directory Documentation

4.1 webP/static Directory Reference

Files

- file [style.css](#)

4.2 webP/templates Directory Reference

Files

- file [index.html](#)

4.3 webP Directory Reference

Directories

- directory [static](#)
- directory [templates](#)

Files

- file [app.py](#)

Chapter 5

Namespace Documentation

5.1 app Namespace Reference

Functions

- [index](#) ()

Variables

- [app](#) = Flask(__name__)
- str [DB_NAME](#) = "../fermentacion.db"
Ruta de la base de datos SQLite.
- [host](#)
- [port](#)
- [debug](#)

5.1.1 Detailed Description

```
@file app.py
@brief Aplicación web Flask para visualización de datos del sistema.
```

Este módulo implementa la interfaz web del sistema de monitoreo de fermentación de café.

Funciones principales:

- Consulta de datos almacenados en SQLite.
- Visualización de variables actuales.
- Generación de históricos para gráficas.
- Renderizado de la página principal mediante Flask.

Variables visualizadas:

- Temperatura interna
- pH
- CO2
- Temperatura exterior
- Humedad exterior

Flujo de operación:

SQLite -> Flask -> Navegador Web

```
@author
Fernando Plazas Trujillo
```

```
@date 2026
```

5.1.2 Function Documentation

5.1.2.1 index()

index ()

@brief Genera la página principal del sistema.

Consulta la base de datos SQLite para obtener:

- Último registro interior.
- Último registro exterior.
- Históricos de temperatura interna.
- Históricos de temperatura exterior.
- Históricos de pH.
- Históricos de CO2.
- Históricos de humedad exterior.

Los datos obtenidos son enviados a la plantilla HTML para su representación gráfica.

@return Página HTML renderizada.

5.1.3 Variable Documentation

5.1.3.1 app

app.app = Flask(__name__)

5.1.3.2 DB_NAME

str app.DB_NAME = "../fermentacion.db"

Ruta de la base de datos SQLite.

5.1.3.3 debug

app.debug

5.1.3.4 host

app.host

5.1.3.5 port

app.port

5.2 mqtt_sqlite Namespace Reference

Functions

- `guardar_exterior` (data)
- `guardar_interior` (data)
- `on_connect` (client, userdata, flags, rc, properties=None)
- `on_message` (client, userdata, msg)

Variables

- str `BROKER` = "localhost"
Dirección del broker MQTT.
- int `PORT` = 1883
Puerto MQTT estándar.
- str `TOPIC_EXTERIOR` = "fermentador/espExterior/datos"
Tópico MQTT utilizado por el ESP32 exterior.
- str `TOPIC_INTERIOR` = "fermentador/espInterior/datos"
Tópico MQTT utilizado por el ESP32 interior.
- str `DB_NAME` = "fermentacion.db"
Nombre de la base de datos SQLite.
- `cliente` = mqtt.Client()
Cliente MQTT principal.
- `on_connect`
Callback de conexión.
- `on_message`
Callback de recepción de mensajes.

5.2.1 Detailed Description

```
@file mqtt_sqlite.py
@brief Servicio MQTT para almacenamiento de datos en SQLite.
```

Este módulo implementa un cliente MQTT que se conecta al broker Mosquitto local de la Raspberry Pi para recibir los datos enviados por los ESP32 del sistema de fermentación.

Los mensajes recibidos son procesados y almacenados en una base de datos SQLite para mantener un registro histórico de las variables monitoreadas.

```
Tópicos gestionados:
- fermentador/espExterior/datos
- fermentador/espInterior/datos
```

Flujo de operación:

```
ESP32 Exterior ----\
                      \ MQTT Broker --> mqtt_sqlite.py --> SQLite
                      /
ESP32 Interior -----/
```

```
@author
Fernando Plazas Trujillo
```

```
@date 2026
```

5.2.2 Function Documentation

5.2.2.1 guardar_exterior()

```
guardar_exterior (  
    data)  
  
@brief Almacena datos provenientes del ESP32 exterior.  
  
Inserta en la tabla 'exterior' los valores de temperatura  
y humedad ambiente junto con su marca temporal.  
  
@param data Diccionario recibido desde MQTT con los datos  
        del sensor exterior.  
  
Formato esperado:  
  
{  
    "timestamp": "YYYY-MM-DD HH:MM:SS",  
    "temp": 25.3,  
    "hum": 70.2  
}  
  
@return None
```

5.2.2.2 guardar_interior()

```
guardar_interior (  
    data)  
  
@brief Almacena datos provenientes del ESP32 interior.  
  
Inserta en la tabla 'interior' las variables asociadas al  
proceso de fermentación.  
  
Variables registradas:  
- Temperatura interna  
- pH  
- Concentración de CO2  
- Valor RAW del sensor MQ-135  
- Voltaje de batería  
- Corriente consumida  
  
@param data Diccionario recibido desde MQTT.  
  
@return None
```

5.2.2.3 on_connect()

```
on_connect (  
    client,  
    userdata,  
    flags,  
    rc,  
    properties = None)
```

@brief Callback ejecutado al establecer conexión MQTT.

Una vez conectado al broker, el cliente se suscribe a los tópicos utilizados por los ESP32 para la transmisión de datos.

@param client Instancia del cliente MQTT.
@param userdata Datos privados asociados al cliente.
@param flags Indicadores de estado de conexión.
@param rc Código de resultado de conexión.
@param properties Propiedades MQTT v5.

@return None

5.2.2.4 on_message()

```
on_message (  
    client,  
    userdata,  
    msg)
```

@brief Procesa los mensajes MQTT recibidos.

Decodifica el payload JSON recibido y determina el tópico de origen para almacenar los datos en la tabla correspondiente.

Tópicos soportados:
- TOPIC_EXTERIOR
- TOPIC_INTERIOR

@param client Instancia del cliente MQTT.
@param userdata Datos privados asociados al cliente.
@param msg Mensaje MQTT recibido.

@return None

5.2.3 Variable Documentation

5.2.3.1 BROKER

```
str mqtt_sqlite.BROKER = "localhost"
```

Dirección del broker MQTT.

5.2.3.2 cliente

```
mqtt_sqlite.cliente = mqtt.Client()
```

Cliente MQTT principal.

5.2.3.3 DB_NAME

```
str mqtt_sqlite.DB_NAME = "fermentacion.db"
```

Nombre de la base de datos SQLite.

5.2.3.4 on_connect

```
mqtt_sqlite.on_connect
```

Callback de conexión.

5.2.3.5 on_message

```
mqtt_sqlite.on_message
```

Callback de recepción de mensajes.

5.2.3.6 PORT

```
int mqtt_sqlite.PORT = 1883
```

Puerto MQTT estándar.

5.2.3.7 TOPIC_EXTERIOR

```
str mqtt_sqlite.TOPIC_EXTERIOR = "fermentador/espExterior/datos"
```

Tópico MQTT utilizado por el ESP32 exterior.

5.2.3.8 TOPIC_INTERIOR

```
str mqtt_sqlite.TOPIC_INTERIOR = "fermentador/espInterior/datos"
```

Tópico MQTT utilizado por el ESP32 interior.

5.3 thingspeak_uploader Namespace Reference

Functions

- [obtener_ultimos_datos](#) ()
- [enviar_thingspeak](#) (interior, exterior)

Variables

- str `DB_NAME` = "fermentacion.db"
Nombre de la base de datos SQLite.
- str `THINGSPEAK_API_KEY` = "..."
Clave de escritura del canal ThingSpeak.
- str `THINGSPEAK_URL` = "https://api.thingspeak.com/update"
URL del servicio de actualización ThingSpeak.
- int `ultimo_id_interior` = 0
Último ID enviado desde la tabla interior.
- int `ultimo_id_exterior` = 0
Último ID enviado desde la tabla exterior.
- `interior`
- `exterior`
- `id_interior` = `interior[0]`
- `id_exterior` = `exterior[0]`
- int `interior_nuevo` = `id_interior` > `ultimo_id_interior`
- int `exterior_nuevo` = `id_exterior` > `ultimo_id_exterior`
- `codigo`

5.3.1 Detailed Description

```
@file thingspeak_uploader.py
@brief Servicio de sincronización entre SQLite y ThingSpeak.

Este módulo obtiene periódicamente los registros más recientes
almacenados en la base de datos SQLite y los publica en la
plataforma ThingSpeak para monitoreo remoto.

Variables enviadas:

- Temperatura interna
- pH
- Concentración de CO2
- Temperatura exterior
- Humedad exterior

Flujo de operación:

SQLite -> thingspeak_uploader.py -> ThingSpeak Cloud

@author
Fernando Plazas Trujillo

@date 2026
```

5.3.2 Function Documentation

5.3.2.1 enviar_thingspeak()

```
enviar_thingspeak (
    interior,
    exterior)

@brief Envía datos a la plataforma ThingSpeak.

Construye el payload HTTP y realiza una solicitud POST
al canal configurado de ThingSpeak.

@param interior Registro más reciente de la tabla interior.
@param exterior Registro más reciente de la tabla exterior.

@return Código HTTP devuelto por el servidor.
```

5.3.2.2 obtener_ultimos_datos()

```
obtener_ultimos_datos ()
```

@brief Obtiene los registros más recientes de la base de datos.

Consulta las tablas interior y exterior y recupera el último registro almacenado en cada una.

@return Tupla (interior, exterior) con los registros obtenidos.

5.3.3 Variable Documentation

5.3.3.1 codigo

```
thingspeak_uploader.codigo
```

Initial value:

```
=  enviar_thingspeak(  
    interior,  
    exterior  
)
```

5.3.3.2 DB_NAME

```
str thingspeak_uploader.DB_NAME = "fermentacion.db"
```

Nombre de la base de datos SQLite.

5.3.3.3 exterior

```
thingspeak_uploader.exterior
```

5.3.3.4 exterior_nuevo

```
int thingspeak_uploader.exterior_nuevo = id_exterior > ultimo_id_exterior
```

5.3.3.5 id_exterior

```
thingspeak_uploader.id_exterior = exterior[0]
```

5.3.3.6 id_interior

```
thingspeak_uploader.id_interior = interior[0]
```

5.3.3.7 interior

```
thingspeak_uploader.interior
```

5.3.3.8 interior_nuevo

```
int thingspeak_uploader.interior_nuevo = id_interior > ultimo_id_interior
```

5.3.3.9 THINGSPEAK_API_KEY

```
str thingspeak_uploader.THINGSPEAK_API_KEY = "..."
```

Clave de escritura del canal ThingSpeak.

5.3.3.10 THINGSPEAK_URL

```
str thingspeak_uploader.THINGSPEAK_URL = "https://api.thingspeak.com/update"
```

URL del servicio de actualización ThingSpeak.

5.3.3.11 ultimo_id_exterior

```
thingspeak_uploader.ultimo_id_exterior = 0
```

Último ID enviado desde la tabla exterior.

5.3.3.12 ultimo_id_interior

```
thingspeak_uploader.ultimo_id_interior = 0
```

Último ID enviado desde la tabla interior.

Chapter 6

File Documentation

6.1 mqtt_sqlite.py File Reference

Namespaces

- namespace [mqtt_sqlite](#)

Functions

- [mqtt_sqlite.guardar_exterior](#) (data)
- [mqtt_sqlite.guardar_interior](#) (data)
- [mqtt_sqlite.on_connect](#) (client, userdata, flags, rc, properties=None)
- [mqtt_sqlite.on_message](#) (client, userdata, msg)

Variables

- str [mqtt_sqlite.BROKER](#) = "localhost"
Dirección del broker MQTT.
- int [mqtt_sqlite.PORT](#) = 1883
Puerto MQTT estándar.
- str [mqtt_sqlite.TOPIC_EXTERIOR](#) = "fermentador/espExterior/datos"
Tópico MQTT utilizado por el ESP32 exterior.
- str [mqtt_sqlite.TOPIC_INTERIOR](#) = "fermentador/esplInterior/datos"
Tópico MQTT utilizado por el ESP32 interior.
- str [mqtt_sqlite.DB_NAME](#) = "fermentacion.db"
Nombre de la base de datos SQLite.
- [mqtt_sqlite.cliente](#) = mqtt.Client()
Cliente MQTT principal.
- [mqtt_sqlite.on_connect](#)
Callback de conexión.
- [mqtt_sqlite.on_message](#)
Callback de recepción de mensajes.

6.2 thingspeak_uploader.py File Reference

Namespaces

- namespace [thingspeak_uploader](#)

Functions

- [thingspeak_uploader.obtener_ultimos_datos](#) ()
- [thingspeak_uploader.enviar_thingspeak](#) (interior, exterior)

Variables

- str [thingspeak_uploader.DB_NAME](#) = "fermentacion.db"
Nombre de la base de datos SQLite.
- str [thingspeak_uploader.THINGSPEAK_API_KEY](#) = "..."
Clave de escritura del canal ThingSpeak.
- str [thingspeak_uploader.THINGSPEAK_URL](#) = "https://api.thingspeak.com/update"
URL del servicio de actualización ThingSpeak.
- int [thingspeak_uploader.ultimo_id_interior](#) = 0
Último ID enviado desde la tabla interior.
- int [thingspeak_uploader.ultimo_id_exterior](#) = 0
Último ID enviado desde la tabla exterior.
- [thingspeak_uploader.interior](#)
- [thingspeak_uploader.exterior](#)
- [thingspeak_uploader.id_interior](#) = interior[0]
- [thingspeak_uploader.id_exterior](#) = exterior[0]
- int [thingspeak_uploader.interior_nuevo](#) = id_interior > ultimo_id_interior
- int [thingspeak_uploader.exterior_nuevo](#) = id_exterior > ultimo_id_exterior
- [thingspeak_uploader.codigo](#)

6.3 webP/app.py File Reference

Namespaces

- namespace [app](#)

Functions

- [app.index](#) ()

Variables

- [app.app](#) = Flask(__name__)
- str [app.DB_NAME](#) = "../fermentacion.db"
Ruta de la base de datos SQLite.
- [app.host](#)
- [app.port](#)
- [app.debug](#)

6.4 webP/static/style.css File Reference

6.5 webP/templates/index.html File Reference

Index

app, [9](#)
 app, [10](#)
 DB_NAME, [10](#)
 debug, [10](#)
 host, [10](#)
 index, [10](#)
 port, [10](#)

BROKER
 mqtt_sqlite, [13](#)

cliente
 mqtt_sqlite, [13](#)

codigo
 thingspeak_uploader, [16](#)

DB_NAME
 app, [10](#)
 mqtt_sqlite, [13](#)
 thingspeak_uploader, [16](#)

debug
 app, [10](#)

enviar_thingspeak
 thingspeak_uploader, [15](#)

exterior
 thingspeak_uploader, [16](#)

exterior_nuevo
 thingspeak_uploader, [16](#)

guardar_exterior
 mqtt_sqlite, [12](#)

guardar_interior
 mqtt_sqlite, [12](#)

host
 app, [10](#)

id_exterior
 thingspeak_uploader, [16](#)

id_interior
 thingspeak_uploader, [16](#)

index
 app, [10](#)

interior
 thingspeak_uploader, [16](#)

interior_nuevo
 thingspeak_uploader, [17](#)

mqtt_sqlite, [11](#)
 BROKER, [13](#)
 cliente, [13](#)
 DB_NAME, [13](#)
 guardar_exterior, [12](#)
 guardar_interior, [12](#)
 on_connect, [12](#), [13](#)
 on_message, [13](#), [14](#)
 PORT, [14](#)
 TOPIC_EXTERIOR, [14](#)
 TOPIC_INTERIOR, [14](#)
mqtt_sqlite.py, [19](#)

obtener_ultimos_datos
 thingspeak_uploader, [15](#)

on_connect
 mqtt_sqlite, [12](#), [13](#)

on_message
 mqtt_sqlite, [13](#), [14](#)

PORT
 mqtt_sqlite, [14](#)

port
 app, [10](#)

THINGSPEAK_API_KEY
 thingspeak_uploader, [17](#)

thingspeak_uploader, [14](#)
 codigo, [16](#)
 DB_NAME, [16](#)
 enviar_thingspeak, [15](#)
 exterior, [16](#)
 exterior_nuevo, [16](#)
 id_exterior, [16](#)
 id_interior, [16](#)
 interior, [16](#)
 interior_nuevo, [17](#)
 obtener_ultimos_datos, [15](#)
 THINGSPEAK_API_KEY, [17](#)
 THINGSPEAK_URL, [17](#)
 ultimo_id_exterior, [17](#)
 ultimo_id_interior, [17](#)
thingspeak_uploader.py, [20](#)

THINGSPEAK_URL
 thingspeak_uploader, [17](#)

TOPIC_EXTERIOR
 mqtt_sqlite, [14](#)

TOPIC_INTERIOR
 mqtt_sqlite, [14](#)

ultimo_id_exterior
 thingspeak_uploader, [17](#)

ultimo_id_interior
 thingspeak_uploader, [17](#)

webP Directory Reference, [7](#)
webP/app.py, [20](#)
webP/static Directory Reference, [7](#)
webP/static/style.css, [20](#)
webP/templates Directory Reference, [7](#)
webP/templates/index.html, [20](#)